In [2]:
```
import warnings
warnings.filterwarnings("ignore")
```

Data Preprocessing:

The text dataset is processed and transformed, to use it for model training. Steps:

- Loading Dataset
- Initialising a tokenizer object for the pretrained model of BERT Transformer
- The transformations on dataset involve
  - Encoding the words with their respective word 'id' from the BERT Model
  - The sentences are padded to make all the sentences of same length in our case the max_length = 200
  - Respective attention vectors are created for each sentence with value '1' for word and value '0' for padded words.
- This transformed requires to have a type of tensorflow tensor data. So we will transform the data into Tensorflow tensor data format.

In [3]:
```
from datasets import load_dataset

dataset = load_dataset("ag_news")
split = dataset['train'].train_test_split(test_size=0.1, seed=42)
train_ds = split["train"]
val_ds = split['test']
test_ds = dataset['test']
```

```
README.md: 0.00B [00:00, ?B/s]
data/train-00000-of-00001.parquet:   0%|          | 0.00/18.6M [00:00<?, ?B/s]
data/test-00000-of-00001.parquet:   0%|          | 0.00/1.23M [00:00<?, ?B/s]
Generating train split:   0%|          | 0/120000 [00:00<?, ? examples/s]
Generating test split:   0%|          | 0/7600 [00:00<?, ? examples/s]
```

In [4]:
```
from transformers import AutoTokenizer

model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

In [5]:
```
def encode_batch(batch):
    return tokenizer(
        batch['text'],
        max_length = 200,
        truncation = True,
        padding = "max_length"
    )

train_enc = train_ds.map(encode_batch, batched=True)
test_enc = test_ds.map(encode_batch, batched=True)
val_enc = val_ds.map(encode_batch, batched=True)
```

```
train_enc = train_enc.rename_column("label", "labels")
test_enc = test_enc.rename_column("label", "labels")
val_enc = val_enc.rename_column("label", "labels")


train_enc.set_format(type = 'tensorflow', columns = ['input_ids','attention_mask', 
test_enc.set_format(type = 'tensorflow', columns = ['input_ids','attention_mask', '
val_enc.set_format(type = 'tensorflow', columns = ['input_ids','attention_mask', 'l
```

```
Map:   0%|          | 0/108000 [00:00<?, ? examples/s]
Map:   0%|          | 0/7600 [00:00<?, ? examples/s]
Map:   0%|          | 0/12000 [00:00<?, ? examples/s]
```

```
2025-11-30 18:01:16.670672: E external/local_xla/xla/stream_executor/cuda/cuda_fft.c
c:477] Unable to register cuFFT factory: Attempting to register factory for plugin c
uFFT when one has already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to STDE
RR
E0000 00:00:1764525676.850106      47 cuda_dnn.cc:8310] Unable to register cuDNN fac
tory: Attempting to register factory for plugin cuDNN when one has already been regi
stered
E0000 00:00:1764525676.904132      47 cuda_blas.cc:1418] Unable to register cuBLAS f
actory: Attempting to register factory for plugin cuBLAS when one has already been r
egistered
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
AttributeError: 'MessageFactory' object has no attribute 'GetPrototype'
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
AttributeError: 'MessageFactory' object has no attribute 'GetPrototype'
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
AttributeError: 'MessageFactory' object has no attribute 'GetPrototype'
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
AttributeError: 'MessageFactory' object has no attribute 'GetPrototype'
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
AttributeError: 'MessageFactory' object has no attribute 'GetPrototype'
```

In [6]:

```
import tensorflow as tf

tf_train = train_enc.to_tf_dataset(
    columns=["input_ids", "attention_mask"],
    label_cols=["labels"],
    shuffle=False,
    batch_size=64,
)


tf_val = val_enc.to_tf_dataset(
    columns=["input_ids","attention_mask"],
    label_cols=["labels"],
    shuffle=False,
    batch_size=64,
)
```

```
tf_test = test_enc.to_tf_dataset(
    columns=["input_ids", "attention_mask"],
    label_cols=["labels"],
    shuffle=False,
    batch_size=64,
)
```

```
I0000 00:00:1764525697.285670      47 gpu_device.cc:2022] Created device /job:localh
ost/replica:0/task:0/device:GPU:0 with 13942 MB memory:  -> device: 0, name: Tesla T
4, pci bus id: 0000:00:04.0, compute capability: 7.5
I0000 00:00:1764525697.286334      47 gpu_device.cc:2022] Created device /job:localh
ost/replica:0/task:0/device:GPU:1 with 13942 MB memory:  -> device: 1, name: Tesla T
4, pci bus id: 0000:00:05.0, compute capability: 7.5
```

Model Initialization

For the text classification task, we employ the BERT Base Uncased model, which is a pretrained Transformer-based encoder. BERT is a bidirectional autoencoder architecture trained on large-scale corpora using two objectives:

- Masked Language Modeling (MLM) — learns deep contextual word representations
- Next Sentence Prediction (NSP) — learns relationships between sentences

Because of this pretraining, BERT produces rich semantic embeddings for every token and for the entire sentence through the [CLS] embedding.

How BERT is used in our classification task

For sequence (text) classification, we follow the standard fine-tuning setup:

- The input text is tokenized into word-piece tokens and converted to integer IDs.
- These token IDs and their attention masks are passed into BERT's encoder stack.
- BERT produces a contextual embedding for every token and a special [CLS] embedding, which summarizes the meaning of the entire input sequence.

On top of this [CLS] embedding, we place a fully connected classification layer:

$$ \mathbf{h}_{\text{CLS}} \in \mathbb{R}^{768} \;\longrightarrow\; \text{Dense}(4) $$

where 4 corresponds to the four AG News categories.

This classification layer is randomly initialized, and only its weights are trained from scratch, while BERT's encoder weights are fine-tuned during training to adapt the representations to our dataset.

What the model learns

- BERT provides context-aware embeddings that capture syntax, semantics, and long-range dependencies.
- The classification head learns to map these embeddings to a 4-class probability distribution using a softmax activation.
- Training optimizes the cross-entropy loss between predicted and true labels.

In [11]:
```
import tensorflow as tf
from transformers import TFBertModel

strategy = tf.distribute.MirroredStrategy()
print("GPUs available:", strategy.num_replicas_in_sync)
```

```
INFO:tensorflow:Using MirroredStrategy with devices ('/job:localhost/replica:0/task:0/device:GPU:0', '/job:localhost/replica:0/task:0/device:GPU:1')
GPUs available: 2
```

In [12]:
```
from tensorflow.keras.layers import Input, Dense, Dropout
from tensorflow.keras.models import Model

max_len = 200

with strategy.scope():

    # Load BERT
    bert = TFBertModel.from_pretrained("bert-base-uncased")

    # Inputs
    input_ids = Input(shape=(max_len,), dtype=tf.int32, name="input_ids")
    attention_mask = Input(shape=(max_len,), dtype=tf.int32, name="attention_mask")

    # Your exact LAMBDA LAYER
    bert_output = tf.keras.layers.Lambda(
        lambda x: bert(x["input_ids"], attention_mask=x["attention_mask"])[0],
        output_shape=(max_len, 768)
    )(
        {"input_ids": input_ids, "attention_mask": attention_mask}
    )

    # CLS token
    cls_embedding = bert_output[:, 0, :]

    # Classification head
    x = Dropout(0.2)(cls_embedding)
    x = Dense(128, activation="relu")(x)
    output = Dense(4, activation="softmax")(x)

    # Build & compile model
    model = Model(inputs=[input_ids, attention_mask], outputs=output)

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
        loss="sparse_categorical_crossentropy",
```

```
        metrics=["accuracy"]
    )

# Show model summary
model.summary()
```

```
model.safetensors:   0%|          | 0.00/440M [00:00<?, ?B/s]
```

```
Some weights of the PyTorch model were not used when initializing the TF 2.0 model T
FBertModel: ['cls.predictions.transform.dense.bias', 'cls.predictions.bias', 'cls.se
q_relationship.weight', 'cls.seq_relationship.bias', 'cls.predictions.transform.Laye
rNorm.bias', 'cls.predictions.transform.dense.weight', 'cls.predictions.transform.La
yerNorm.weight']
- This IS expected if you are initializing TFBertModel from a PyTorch model trained
on another task or with another architecture (e.g. initializing a TFBertForSequenceC
lassification model from a BertForPreTraining model).
- This IS NOT expected if you are initializing TFBertModel from a PyTorch model that
you expect to be exactly identical (e.g. initializing a TFBertForSequenceClassificat
ion model from a BertForSequenceClassification model).
All the weights of TFBertModel were initialized from the PyTorch model.
If your task is similar to the task the model of the checkpoint was trained on, you
can already use TFBertModel for predictions without further training.
```

Model: "functional_1"

Layer (type)	Output Shape	Param #	Connected to
attention_mask (InputLayer)	(None, 200)	0	-
input_ids (InputLayer)	(None, 200)	0	-
lambda_1 (Lambda)	(None, 200, 768)	0	attention_mask[0… input_ids[0][0]
get_item_1 (GetItem)	(None, 768)	0	lambda_1[0][0]
dropout_1 (Dropout)	(None, 768)	0	get_item_1[0][0]
dense_2 (Dense)	(None, 128)	98,432	dropout_1[0][0]
dense_3 (Dense)	(None, 4)	516	dense_2[0][0]

Total params: 98,948 (386.52 KB)

Trainable params: 98,948 (386.52 KB)

Non-trainable params: 0 (0.00 B)

In [10]:
```
import tensorflow as tf
print(tf.config.list_physical_devices('GPU'))
```

```
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU'), PhysicalDevice(na
me='/physical_device:GPU:1', device_type='GPU')]
```

In [ ]:
```
%%time
import time
cpu_start_time = time.process_time()

history = model.fit(
    tf_train,
    validation_data = tf_val,
    epochs = 10
)
cpu_end_time = time.process_time()
cpu_train_time = cpu_end_time - cpu_start_time
print(cpu_train_time)
```

```
INFO:tensorflow:Reduce to /job:localhost/replica:0/task:0/device:CPU:0 then broadcas
t to ('/job:localhost/replica:0/task:0/device:CPU:0',).
INFO:tensorflow:Reduce to /job:localhost/replica:0/task:0/device:CPU:0 then broadcas
t to ('/job:localhost/replica:0/task:0/device:CPU:0',).
INFO:tensorflow:Reduce to /job:localhost/replica:0/task:0/device:CPU:0 then broadcas
t to ('/job:localhost/replica:0/task:0/device:CPU:0',).
Epoch 1/10
INFO:tensorflow:Collective all_reduce tensors: 1 all_reduces, num_devices = 2, group
_size = 2, implementation = CommunicationImplementation.NCCL, num_packs = 1
INFO:tensorflow:Collective all_reduce tensors: 1 all_reduces, num_devices = 2, group
_size = 2, implementation = CommunicationImplementation.NCCL, num_packs = 1
INFO:tensorflow:Collective all_reduce tensors: 4 all_reduces, num_devices = 2, group
_size = 2, implementation = CommunicationImplementation.NCCL, num_packs = 1
INFO:tensorflow:Collective all_reduce tensors: 1 all_reduces, num_devices = 2, group
_size = 2, implementation = CommunicationImplementation.NCCL, num_packs = 1
INFO:tensorflow:Collective all_reduce tensors: 1 all_reduces, num_devices = 2, group
_size = 2, implementation = CommunicationImplementation.NCCL, num_packs = 1
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 0s 465ms/step - accuracy: nan - loss: nanINFO:tensorf
low:Reduce to /job:localhost/replica:0/task:0/device:CPU:0 then broadcast to ('/job:
localhost/replica:0/task:0/device:CPU:0',).
INFO:tensorflow:Reduce to /job:localhost/replica:0/task:0/device:CPU:0 then broadcas
t to ('/job:localhost/replica:0/task:0/device:CPU:0',).
INFO:tensorflow:Reduce to /job:localhost/replica:0/task:0/device:CPU:0 then broadcas
t to ('/job:localhost/replica:0/task:0/device:CPU:0',).
INFO:tensorflow:Collective all_reduce tensors: 1 all_reduces, num_devices = 2, group
_size = 2, implementation = CommunicationImplementation.NCCL, num_packs = 1
INFO:tensorflow:Collective all_reduce tensors: 1 all_reduces, num_devices = 2, group
_size = 2, implementation = CommunicationImplementation.NCCL, num_packs = 1
INFO:tensorflow:Collective all_reduce tensors: 1 all_reduces, num_devices = 2, group
_size = 2, implementation = CommunicationImplementation.NCCL, num_packs = 1
INFO:tensorflow:Collective all_reduce tensors: 1 all_reduces, num_devices = 2, group
_size = 2, implementation = CommunicationImplementation.NCCL, num_packs = 1
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 894s 521ms/step - accuracy: nan - loss: nan - val_acc
uracy: nan - val_loss: nan
Epoch 2/10
1145/1688 ━━━━━━━━━━━━━━━━━━━━ 4:13 467ms/step - accuracy: nan - loss: nan
```

Model Evaluation:

We will check the model on training and test dataset and collect various model metrics, They are:

- Train Dataset Metrics
- Test Dataset Metrics
- Metrics used:
  - Accuracy
  - F1 Score
- Inference Time of Test Dataset
- FLOP's for each Sample Inference

In [ ]:
```
model.save("/kaggle/working/my_model.keras")
tokenizer.save_pretrained('tokenizer')
```

In [ ]:
```
%%time
import time
import numpy as np


# Start CPU timer
cpu_start_time = time.process_time()
start_wall = time.time()

# Run prediction
y_pred_probs = model.predict(tf_test
)

y_test_pred = np.argmax(y_pred_probs, axis=1)

# Stop timers
cpu_end_time = time.process_time()
end_wall = time.time()

cpu_time_inf = cpu_end_time - cpu_start_time
wall_time_inf = end_wall - start_wall

print("CPU inference time:", cpu_time_inf, "seconds")
print("Wall-clock inference time:", wall_time_inf, "seconds")
```

In [ ]:
```
y_train_probs = model.predict(tf_train)

y_train_pred = np.argmax(y_train_probs, axis=1)
```

In [ ]:
```
y_train = train_enc['labels']
```

In [ ]:
```
from sklearn.metrics import accuracy_score,f1_score, classification_report
print("Training Accuracy")
train_accuracy = accuracy_score(y_train, y_train_pred)
train_f1 = f1_score(y_train, y_train_pred, average='weighted', zero_division=0)
train_report = classification_report(y_train, y_train_pred)
print('\n Training Accuracy',train_accuracy)
print('\n Training F1 Score',train_f1)
print('\n Training Classification Report',train_report)
```

In [ ]:
```
y_test = test_enc['labels']
```

In [ ]:
```
print("Test Accuracy")
test_accuracy = accuracy_score(y_test, y_test_pred)
test_f1 = f1_score(y_test, y_test_pred, average='weighted', zero_division=0)
test_report = classification_report(y_test,y_test_pred)
print('\n Test Accuracy',test_accuracy)
print('\n Test F1 Score',test_f1)
print('\n Test Classification Report',test_report)
```

FLOP's Calculation:

A Single BERT Layer includes:

- Q, K, V projections → 3 matrix multiplications
- Attention softmax
- Attention × V
- Feed-forward network (two dense layers)
- Layer norm, dropout, residuals

A 12-layer BERT-base model ≈ 22Billion FLOPs per sequence of length 200.

In [ ]:
```
flops = 22_000_000_000
```

In [ ]:
```
results = {}
results[('BERT Transformer','MLP Model')] = {
                                        'Train':
                                               {'accuracy': train_accuracy,
                                               'f1_score': train_f1,
                                               'report': train_report,
                                               'training_time': cpu_train_time},
                                        'Test':
                                               {'accuracy': test_accuracy,
                                               'f1_score': test_f1,
                                               'report': test_report,
                                               'inference_time':cpu_time_inf,
                                               "flops/sample": flops}}
```

In [ ]:
```

```

In [ ]:
```

```

Tabulate:

In [ ]:
```
# create DataFrame
import pandas as pd
```

```
# flatten nested dict into a list of rows
rows = []

for (vectorizer, model), metrics in results.items():
    row = {
        "Vectorizer": vectorizer,
        "Model": model,
        "Train_Accuracy": metrics["Train"]["accuracy"],
        "Train_F1": metrics["Train"]["f1_score"],
        "Test_Accuracy": metrics["Test"]["accuracy"],
        "Test_F1": metrics["Test"]["f1_score"],
        "Inference_Time": metrics["Test"].get("inference_time", None),
        "FLOPs_per_Sample": metrics["Test"].get("flops/sample", None),
        "Training_Time": metrics['Train'].get('training_time',None)
    }
    rows.append(row)

# create DataFrame
df_results = pd.DataFrame(rows)

# optional: set a clean display order
df_results = df_results[
    ["Vectorizer", "Model", "Train_Accuracy", "Train_F1",
     "Test_Accuracy", "Test_F1", "Inference_Time", "FLOPs_per_Sample","Training_Tim
]

print(df_results)
```

In [ ]:
```
df_results.to_csv('/kaggle/working/w2vec_emb.csv')
```

In [ ]:
```
df_results
```

In [ ]:
```

```